



軟體工程：讓程式變成可長期運作的系統

軟體工程不是單純「寫程式」，而是用工程化的方法，讓軟體可以被規劃、開發、測試、部署、維護與改善。程式能跑，只是開始；系統能長期穩定運作，才是軟體工程真正關心的事。

軟體工程是什麼？

很多人以為軟體工程就是寫程式，其實寫程式只是其中一個階段。軟體工程關心的是，這個系統能不能在真實世界被使用、被修改、被交接、被維護。程式能跑只是起點，系統能長期穩定運作才是終點。

寫程式

- 功能實作
- 語法與邏輯
- 畫面呈現
- 演算法設計

軟體工程

- 需求分析與確認
- 系統架構設計
- 測試策略與驗證
- 部署、維護與文件
- 風險評估與版本管理
- 長期可交接性

換句話說，寫程式是建造房間裡的家具；軟體工程是確保整棟建築有設計圖、有施工標準、有安全檢查，並且在未來十年仍然可以安全居住。兩者都重要，但層次完全不同。當你面對的是企業級系統，光靠寫程式的思維是不夠的。

軟體不是做完就結束，而是上線後才開始

企業系統通常不是一次性作品，而是會長期使用、持續修改的系統。如果沒有軟體工程的思維與實踐，系統往往在短短幾年內就會陷入難以維護的困境。

需求說不清楚

做出來的功能不是客戶真正要的，反覆修改卻越改越遠。

程式沒有人看得懂

原始開發者離職後，沒有任何文件，新人接手等於重頭來過。

系統一改就壞

牽一髮動全身，維護成本隨著時間指數成長，最終只能忍著不動。

沒有測試保護

錯誤不在開發時被發現，而是在上線後由真實用戶踩雷才爆發。

沒有版本控管

改錯了回不去，多人協作互相覆蓋，程式庫成為一個充滿地雷的黑盒子。



軟體工程的價值，不是在第一天讓你寫得比較快，而是在半年、一年、三年後，系統還能活得下去。

軟體工程的核心：四個關鍵能力

軟體工程主要關心四件事，這四件事決定了一個系統是否真正具備工程品質。如果只是課堂練習，能跑就好；但如果是企業系統，能跑只是基本要求，後面還要面對變更、錯誤、維護、稽核與責任。



可交付

做出來的系統符合需求，可以真正被使用。不是自己覺得做好了，而是客戶與用戶確認可以接受。



可維護

出問題時找得到原因，也有人能接手修改。程式碼有結構、有文件、有測試，不依賴單一開發者的記憶。



可擴充

未來需求改變時，系統不會整個推倒重來。良好的架構設計讓新功能可以被加入，而不破壞既有邏輯。



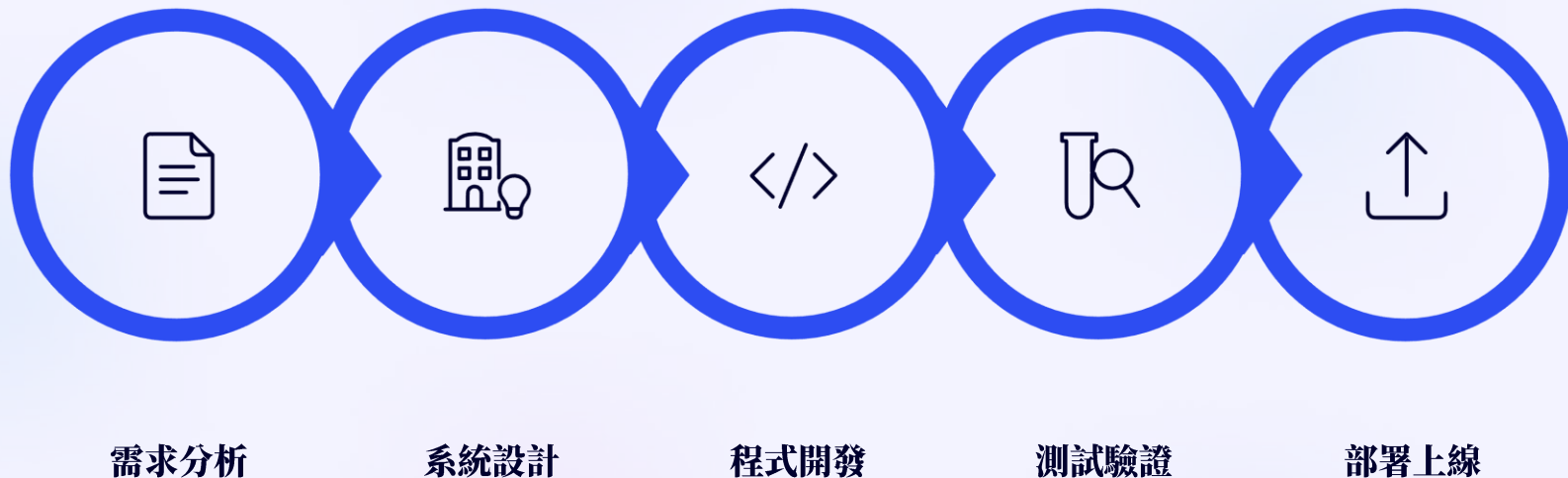
可驗證

系統是否正確，不能只靠感覺，要能透過測試、紀錄與稽核確認。這也是企業合規的基本要求。

這四個能力環環相扣：沒有可驗證，就不知道系統是否可交付；沒有可維護，可擴充就無從談起。真正的軟體工程，是讓這四個維度同時得到照顧。

瀑布式生命週期：一步一步完成軟體開發

瀑布式開發是一種傳統的軟體生命週期模型，也是業界最早被系統化的工程方法之一。它把軟體開發分成幾個明確、有順序的階段，前一個階段完成並確認後，才進入下一個階段。這種方式的最大貢獻，是把「寫程式」放回整個生命週期中，提醒我們開發前要分析需求，上線前要測試，上線後還要維護。



瀑布式模型的每一個階段都有明確的輸入與輸出：需求分析產出需求規格書，系統設計產出架構文件，程式開發產出可執行程式碼，測試驗證產出測試報告，部署上線讓系統進入生產環境，維護改善則持續確保系統的健康狀態。這種清晰的階段劃分，讓專案管理者可以追蹤進度、分配責任、確保品質門檻，是許多大型政府與企業專案至今仍採用的基礎架構。

瀑布式開發：清楚、穩定，但不容易面對變化

瀑布式開發有其獨特的優點，也有明顯的限制。理解這兩面，才能在正確的情境下做出正確的選擇。

✓ 優點

- 階段清楚，進度容易管理
- 文件完整，可追溯性高
- 適合需求穩定的專案
- 責任分工明確
- 適合高度合規、需要稽核的環境
- 減少跨部門溝通摩擦

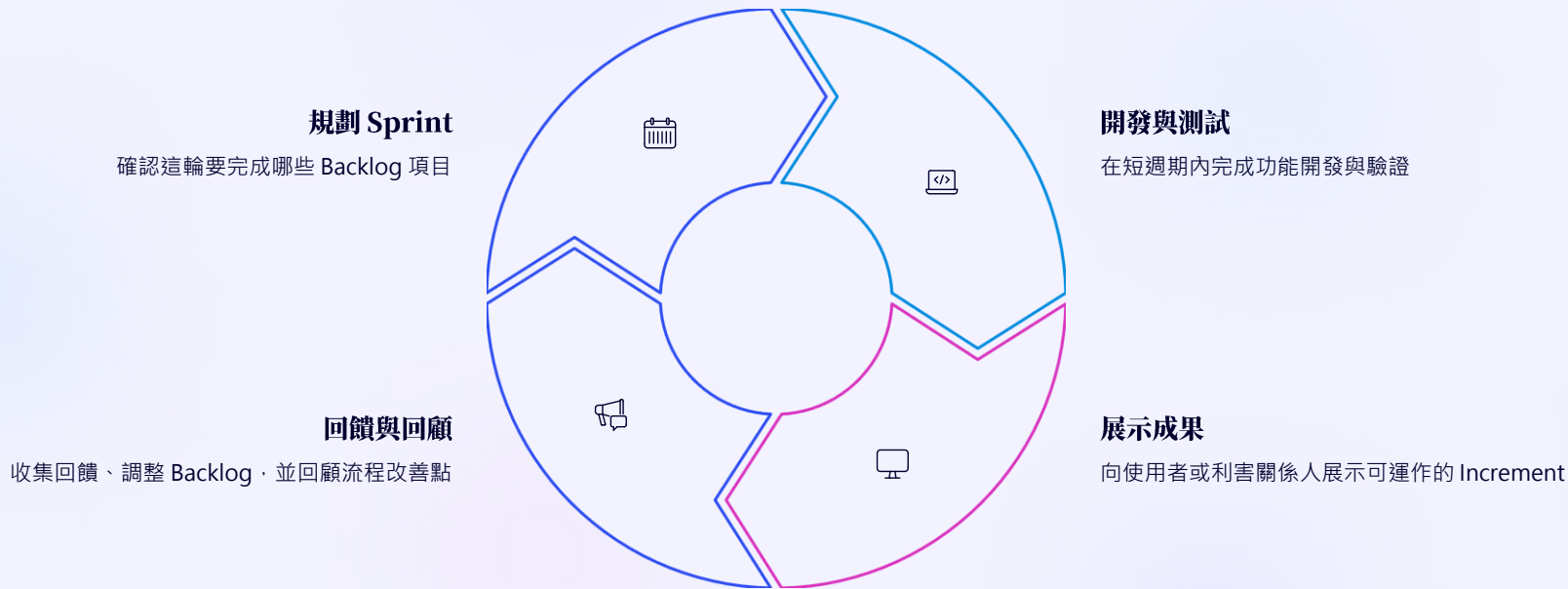
⚠ 限制

- 前期需求若判斷錯誤，後期修改成本極高
- 使用者很晚才看到成品，回饋太慢
- 不適合需求快速變動的專案
- 容易變成文件很多，但系統未必符合現實
- 靈活性低，難以應對市場或技術的快速變化

瀑布式不是落伍的方法，它適合某些特定的環境：需求相對穩定、責任必須清楚、流程需要稽核、變更成本非常高的系統，例如航太、醫療設備、核電廠控制系統等高風險領域。但如果需求一直在變，或者客戶本身也不確定要什麼，瀑布式就會非常吃力——因為它的設計前提就是「你在一開始就能把需求說清楚」。

敏捷式開發：不要等到最後才知道做錯了

敏捷式開發強調短週期、快速交付、持續回饋。它不期待一開始就把所有需求想清楚，而是承認現實世界的需求會變，所以用短週期降低做錯方向的風險。每一輪迭代（Sprint）都產出可運作的成果，讓使用者盡早看到、盡早回饋。



敏捷常見的核心概念包括：**Sprint**（短週期開發，通常 1–4 週）、**Backlog**（待開發需求清單，依優先順序排列）、**User Story**（從使用者角度描述的功能需求）、**Increment**（每次 Sprint 結束交付的可運作成果）、**Review**（檢視成果是否符合期待）、**Retrospective**（回顧流程並找出改善點）。敏捷式開發不是沒有規劃，而是把規劃分散到每個週期，讓方向可以隨時根據真實回饋調整。

瀑布式 vs 敏捷式：不是誰取代誰，而是看情境選擇

真正成熟的軟體工程師，不是只會說瀑布式落伍，或敏捷式最好，而是知道不同專案要選擇不同方法。重點不是方法名稱，而是能不能降低風險、交付價值、讓系統長期活下去。

面向	瀑布式	敏捷式
開發方式	依序完成各階段，前一階段確認後才進入下一階段	短週期反覆迭代，每輪都產出可運作成果
需求處理	前期盡量定義清楚，後期變更成本高	開發過程中逐步調整，擁抱需求變化
使用者回饋	較晚出現，通常在驗收階段才看到成品	持續出現，每個 Sprint 結束就有機會收集
文件完整性	較完整，每個階段有正式輸出文件	較精簡，以可運作軟體取代大量文件
適合情境	需求穩定、合規要求高、大型基礎建設	需求變動、快速驗證、網站/App/創新型專案



軟體工程的真正意義

軟體工程不是讓人寫更多文件，而是讓軟體從「能跑」走向「能用、能改、能維護」。

AI 時代寫程式的速度越來越快，工具越來越強大，但這也意味著我們能在更短的時間內製造出更多技術債。正因如此，軟體工程的重要性不降反升——我們更需要它來管理需求、設計架構、確保測試品質、釐清責任歸屬。

需求管理

確保我們在做對的事，而不只是把事做對

架構設計

讓系統在成長的過程中不會崩潰

測試保護

讓每一次改動都有安全網，錯誤不會悄悄流入生產環境

責任釐清

讓系統可以被交接、被稽核、被長期信任

學習軟體工程，不是為了遵守繁瑣的規則，而是為了讓你寫的每一行程式，都能在真實世界中發揮它應有的價值——不只今天，還有明天、後天、三年後。這是工程師對自己作品應有的承諾，也是專業與業餘的分水嶺。

